

Overview

1 Markov chain Monte Carlo

- Recap
- Autocorrelations
- The heat bath algorithm

2 Minimisation

- One-dimensional minimisation
- Downhill Simplex (Nelder-Mead) algorithm
- Direction set methods in many dimensions
- Global minimisation, simulated annealing
- Summary

Recap

Markov chain Monte Carlo

Generate an ergodic Markov chain of configurations which leads to the **stationary** distribution $P(X) = \exp(-\beta H(X))$, using **detailed balance**:

$$P(X)T(X \rightarrow Y) = P(Y)T(Y \rightarrow X)$$

Write transition probability as $T(X \rightarrow Y) = \omega_{XY}A_{XY}$

- $\omega_{XY} = \omega_{YX}$ = trial step probability, $\sum_Y \omega_{XY} = 1$.
- $A_{XY} \neq A_{YX}$ = acceptance probability

Metropolis algorithm

Use the acceptance probability

$$A_{XY} = \begin{cases} 1 & P(X) \leq P(Y) \\ \frac{P(Y)}{P(X)} & P(X) > P(Y) \end{cases}$$

Autocorrelations

Two successive configurations in Markov chain will be “close” in phase space $\implies$ any quantity will have similar values on the two configs

The two are correlated: $\langle \Phi[X_n]\Phi[X_{n+1}] \rangle \neq \langle \Phi \rangle^2$

For Monte Carlo we need samples to be statistically independent

Autocorrelation time τ

$$R(t) = \langle \Phi[X_n]\Phi[X_{n+t}] \rangle - \langle \Phi \rangle^2 = Ce^{-t/\tau}$$

Measures the “memory” of the Markov process

Configurations separated by $t \gtrsim 2\tau$ are statistically independent

τ can often be “eyeballed” by looking at history plots

Autocorrelation issues

- ① It is often crucial to make the autocorrelation time as short as possible
→ improved simulation algorithms may be needed
- ② Autocorrelations increase dramatically near phase transitions
→ critical slowing down
- ③ Statistical uncertainties are underestimated if autocorrelations are ignored
→ multiply errors by τ or use robust error estimation methods

The heat bath algorithm

One of the alternatives to Metropolis

Algorithm

We update only a few degrees of freedom (eg single site) at a time.
Call these x ; the surrounding 'environment' (rest of system) is $X - x$
Choose new x randomly with distribution

$$P(x) \propto e^{-\beta H(x|X-x)}$$

The new x does not depend on the old x , only on old $X - x$.
Equivalent to an infinite number of Metropolis steps

Drawback We need to write $\exp(-\beta H(x|X-x))$ in closed form and sample this distribution efficiently

Not so bad:

- This distribution is sometimes gaussian.
- For Ising-like models, can write probabilities exactly.
- For Ising, heat bath $\equiv$ Glauber.

Proof of detailed balance

We write $X = \{x, X - x\}$; $Y = \{y, X - x\} \equiv \{y, Y - y\}$.

The transition probability is then given by

$$T(X \rightarrow Y) = Ze^{-\beta H(y|X-x)} = P(y|X-x).$$

Using Bayes' theorem $P(A, B) = P(A|B)P(B)$ we hence have

$$\begin{aligned} P(X)T(X \rightarrow Y) &= P(x, X - x)P(y|X - x) \\ &= P(x|X - x)P(X - x)P(y|X - x), \\ P(Y)T(Y \rightarrow X) &= P(y, X - x)P(x|X - x) \\ &= P(y|X - x)P(X - x)P(x|X - x), \end{aligned}$$

and we see the two expressions are the same.

Minimisation

- Physical problems often mean minimising energy, free energy, action etc
- Data fitting = minimising χ^2
- Other problems may be rewritten as minimisation
 - ▶ matrix equation $Ax = b \Leftrightarrow$ minimise $f(x) = \frac{1}{2}x^\dagger Ax - x^\dagger b$

Two very different problems

- **Local minimum: easy**
 - ▶ Go downhill till you start going uphill again!
 - ▶ Many dimensions: many directions
 - ▶ Issues: speed (and storage): evaluate f as few times as possible
- **Global minimum: extremely hard**
 - ▶ May have ragged function with very many local minima
 - ▶ How sure can we be that we have found **the** minimum?

Local minimisation

[See Numerical Recipes chap 10 for details]

Looks a lot like root finding, but easier:

- 1 Existence of a minimum is more straightforward to prove
- 2 Minimum in many dims is combination of minima in each direction
- 3 There is always a 'downhill' so searches do not get lost

1 dimension

- Equivalent of bracketing a root: Bracketing a minimum requires 3 points $a < b < c : f(b) < f(a) \& f(b) < f(c)$
- Equivalent of bisection: Golden section [NR 10.1]
- Equivalent of false position: Parabolic interpolation / Brent [NR 10.2]

Many dimensions

- Simplex: requires no knowledge of derivatives of directions
- Direction set methods: minimise successively in different directions
Optimal choice of directions is crucial!

Golden section (1-dimensional minimisation)

Bisection: After bracketing a root, subdivide the interval in halves until we have zoomed in on the root.

Bracketing a **minimum** involves finding **three** points.

What is the optimal way of subdividing an interval in three?

Assume we have three points $a < b < c$ such that $a - b < b - c$ and

$$\frac{b-a}{c-a} = w, \quad \frac{c-b}{c-a} = 1-w.$$

We choose a new point x a fraction z into the (larger) interval $\langle b, c \rangle$,

$$\frac{x-b}{c-b} = z.$$

The minimum is now bracketed **either** by (a, b, x) **or** by (b, x, c) .

Golden section

Choosing $x - a = c - b$ and $w = z$ gives us

$$w^2 - 3w + 1 = 0 \quad \implies \quad w = \frac{3 - \sqrt{5}}{2} \approx 0.38192$$

Parabolic interpolation

Like bisection, golden section ignores the values of the function, ie how close you are to the minimum.

The simplest way of estimating the position of the minimum is

fit the three points to a parabola, $f(x) = Ax^2 + Bx + C$

$$Aa^2 + Ba + C = y_1 \quad \& \quad Ab^2 + Bb + C = y_2 \quad \& \quad Ac^2 + Bc + C = y_3$$

$$\implies x_{\min} = -\frac{B}{2A} = b + \frac{1}{2} \frac{(b-a)^2(y_2 - y_3) + (b-c)^2(y_2 - y_1)}{(b-a)(y_2 - y_3) + (b-c)(y_2 - y_1)}$$

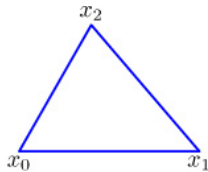
This can go very wrong without further checks:

- We may find a maximum, not a minimum
- We may no longer bracket the minimum
- It can jump around without converging

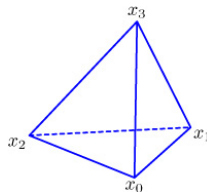
A sure-fire method is [Brent's method](#) [NR 10.2], which switches between golden section and parabolic interpolation as appropriate.

Downhill Simplex (Nelder-Mead) algorithm, a.k.a. amoeba algorithm

For n -variable problem,
choose $n + 1$ points and calculate $f(\mathbf{x})$ at all these points.



Simplex in 2D



Simplex in 3D

‘Move’ the point with largest value of $f(\mathbf{x})$. Repeat. Repeat.

Line minimisation in many dimensions

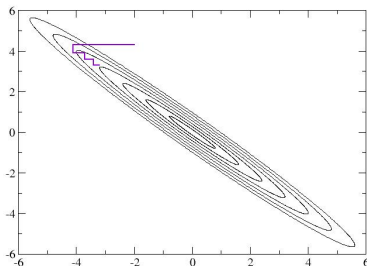
The most obvious way of minimising $f(\mathbf{x}) = f(x_1, x_2, \dots)$ is

- 1 Choose a starting guess $\mathbf{x}^{(0)}$
- 2 Find the minimum in the x_1 -direction, ie
find $\lambda : f(\mathbf{x}^{(0)} + \lambda \mathbf{e}_1) = \min$; take $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \lambda \mathbf{e}_1$
- 3 Minimise in x_2 -direction: $f(\mathbf{x}^{(1)} + \lambda \mathbf{e}_2) = \min$; $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \lambda \mathbf{e}_2$
Repeat for all directions
- 4 Repeat steps 2-3 until we are at a minimum for all directions

i.e., minimize successively in
directions $(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n)$

This should work eventually, but may
be inefficient.

E.g., bad for long, narrow valleys in
diagonal directions



Once in valley, would prefer to head straight to minimum

Steepest descent and Gradient descent

What if we can calculate the gradient, $\nabla f(\mathbf{x})$?

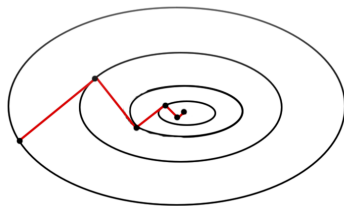
$\nabla f(\mathbf{x})$ is a n -dimensional vector.

$-\nabla f(\mathbf{x})$ is the direction of steepest descent.

Instead of minimizing successively in directions $(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n)$,

Minimize successively in the direction of $-\nabla f(\mathbf{x})$.

- 1 Choose a starting guess $\mathbf{x}^{(0)}$
- 2 Find minimum in direction of $-\nabla f(\mathbf{x}^{(0)})$, ie
find $\lambda = \lambda_m$ minimizing
 $f(\mathbf{x}^{(0)} - \lambda \nabla f(\mathbf{x}^{(0)}))$.
Take $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} - \lambda_m \nabla f(\mathbf{x}^{(0)})$.
- 3 From $\mathbf{x}^{(1)}$, minimise in direction of $-\nabla f(\mathbf{x}^{(1)})$:
- 4 Repeat until minimum is reached.



Successive directions
perpendicular!

Lines stop & turn when
tangential to a contour.

Steepest descent and Gradient descent

Could be easier to just move a bit downward in the direction of $-\nabla f(\mathbf{x})$, successively.

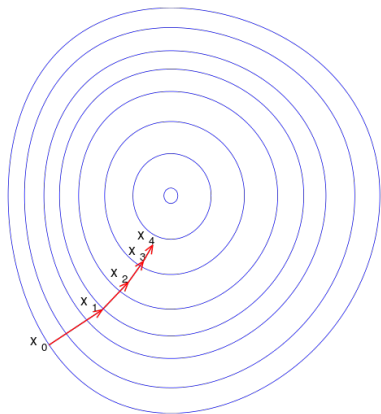
Saves the effort of minimising along each direction —

At the expense of calculating gradient more often, possibly.

Use if gradient calculation is cheap.

Usually called **gradient descent**.

Now popular in machine learning, esp. neural networks.



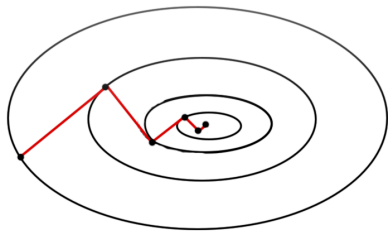
$$\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} - \alpha \nabla f(\mathbf{x}^{(i)})$$

$\alpha \longrightarrow$ **learning rate**

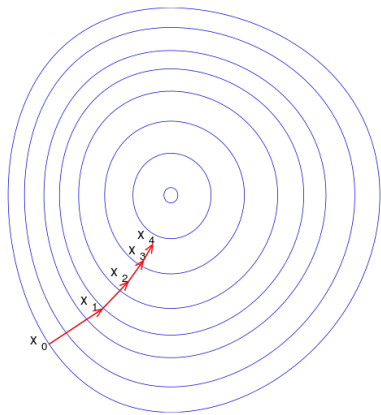
Steepest descent versus Gradient descent

Hard to tell difference from the names. Sometimes used interchangeably.

Usually: **Steepest descent** refers to successive minimizations; **gradient descent** refers to moving 'a little bit each time'.



Steepest descent



Gradient descent

Conjugate directions + Conjugate Gradient Method

Instead of minimising along **perpendicular directions**, e.g., $(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n)$, what if we could choose directions that don't spoil previous minimisations?

→ Choose **conjugate directions**, $(\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n)$.

For quadratic functions, $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\dagger \mathbf{A} \mathbf{x} - \mathbf{x}^\dagger \mathbf{b}$, conjugate directions obey

$$\mathbf{p}_i^\dagger \mathbf{A} \mathbf{p}_j = 0 \quad \text{for } i \neq j$$

(A -orthogonal instead of just orthogonal.)

→ Conjugate gradient method for minimising $\frac{1}{2}\mathbf{x}^\dagger \mathbf{A} \mathbf{x} - \mathbf{x}^\dagger \mathbf{b}$
i.e., method for **solving matrix equation** $\mathbf{A} \mathbf{x} = \mathbf{b}$.

Reaches minimum in n steps.

Minimising arbitrary function?

Conjugate directions + Conjugate Gradient Method

Near a minimum, any function is a quadratic form (Taylor expand):

$$f(\mathbf{x}) \approx f(P) + \sum_i \frac{\partial f}{\partial x_i} x_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} x_i x_j = c - \mathbf{x}^\dagger \mathbf{b} + \frac{1}{2} \mathbf{x}^\dagger H \mathbf{x}$$

(H is known as **Hessian** matrix.)

Functions generally not exactly quadratic; Hessians generally not exactly known.

Nevertheless, (approximate) conjugate gradient works well.

Algorithm is quite involved.

Conjugate gradient — listing of algorithm

Recipe

- 1 Start with arbitrary $\mathbf{r}_0, \mathbf{u}_0 = \mathbf{r}_0$
- 2 Construct $\mathbf{r}_{i+1} = \mathbf{r}_i - \lambda_i \mathbf{H} \cdot \mathbf{u}_i$ with $\lambda_i = \frac{\mathbf{r}_i \cdot \mathbf{r}_i}{\mathbf{u}_i \cdot \mathbf{H} \cdot \mathbf{u}_i}$
- 3 Construct $\mathbf{u}_{i+1} = \mathbf{r}_{i+1} + \gamma_i \mathbf{u}_i$ with $\gamma_i = \frac{\mathbf{r}_{i+1} \cdot \mathbf{r}_{i+1}}{\mathbf{r}_i \cdot \mathbf{r}_i}$

These satisfy

$$\mathbf{r}_i \cdot \mathbf{r}_j = 0, \quad \mathbf{u}_i \cdot \mathbf{H} \cdot \mathbf{u}_j = 0, \quad \mathbf{r}_i \cdot \mathbf{u}_j = 0 \quad \forall j < i$$

- 4 Iterate until it converges

But we don't know $\mathbf{H}$??!

We do not need to:

If $\mathbf{r}_i = \nabla f(\mathbf{x}_i)$ then λ_i minimises $f(\mathbf{x}_i + \lambda \mathbf{u}_i)$ and $\mathbf{r}_{i+1} = \nabla f(\mathbf{x}_{i+1})$:

$$\nabla f(\mathbf{x}_i - \lambda \mathbf{u}_i) = \nabla f(\mathbf{x}) - \lambda \mathbf{H} \cdot \mathbf{u}_i = \mathbf{r}_i - \lambda \mathbf{H} \cdot \mathbf{u}_i$$

$$\mathbf{u}_i \cdot \nabla f(\mathbf{x}_i - \lambda \mathbf{u}_i) = 0 \quad \implies \quad \mathbf{u}_i \cdot \mathbf{r}_i = \mathbf{r}_i \cdot \mathbf{r}_i = -\lambda \mathbf{u}_i \cdot \mathbf{H} \cdot \mathbf{u}_i$$

Global minimisation

Many physical systems (esp complex or disordered ones) have a large number of 'local' minima (metastable states):

- configurations of large molecules (proteins)
- spin glasses
- neural networks
- integrated circuit design (minimise interference)
- travelling salesman:
find the shortest round trip for a salesman visiting N towns

Simulated annealing

Analogy

Cooling or crystallisation of liquids (annealing)

They will settle into their ground states if cooled slowly enough

Simulated annealing

Simulate thermal system with $P(X) \propto \exp(-\beta H(X)) = \exp(-f(X)/T)$

Start with high T , lower it gradually and slowly

We need:

- 1 description of possible states X
- 2 energy function $E = f(X)$ to be minimised
- 3 generator of random changes
- 4 control parameter T and plan for lowering it:
 - ▶ how many random changes to try at each T
 - ▶ how much to lower T each time
- 5 a stopping criterion!

Travelling salesman

- ① states = paths = permutations of numbers $1 \dots N$
- ② $f(X)$ = length of total path
- ③ two types of changes:
 - ① reverse segment $[x_i, x_j]$
 - ② move segment to different place on path
- ④ we need to try this out!!
- ⑤ stop when system is 'frozen' (no change happening)

Summary

Minimisation

- Local minimisation in many dimensions:
 - ▶ **Direction set methods** in many dimensions:
minimise successively in different directions
 - ▶ **Conjugate gradient**:
choose directions which do not spoil previous minimisations
- Global minimisation: **hard** (or impossible!)
 - ▶ **Simulated annealing**:
“cool” the system down slowly till it settles in its ground state